



Program: BS Computer Science

Semester: Fall 2025

Credit Hour(s): 1

Pre-requisite(s): EE1005 Digital Logic Design

Instructor: Engr. Muhammad Hassaan Bin Shoukat – hassaan.shoukat@nu.edu.pk

Lab # 10: Subroutines and Parameter Passing

1. Objectives

- Utilize the PUSH and POP instructions to manipulate data.
- Write and use subroutines with CALL and RET.
- Implement register preservation within subroutines to prevent data loss.
- Pass parameters to subroutines using the stack and access them via the BP register.
- Understand the difference between caller and callee stack cleanup.

2. The Stack

- **Stack:** The stack is a data structure that operates in a "First In, Last Out" (**FILO**) or "Last In, First Out" (**LIFO**) manner.
- **Key Registers:**
 - **SS (Stack Segment):** This register holds the starting address of the memory segment allocated for the stack.
 - **SP (Stack Pointer):** This register holds the offset of the "**top of the stack**" within the Stack Segment. The physical address of the top of the stack is given by the SS:SP combination.
- **Word-Sized Operations:** The 8088 stack works on **word-sized (2-byte)** elements. You cannot push or pop single bytes.

2.1. Basic Stack Operations (PUSH & POP)

The PUSH and POP operations are the primary way to add and remove data from the stack. They are perfect for *temporarily* saving data.

Example 1: Demonstrate use of stack with XCHG Instruction

The XCHG instruction can swap values, but you can also use the stack to achieve the same result.

```
[org 0x0100]
    mov ax, 1234h
    mov bx, 5678h

    push ax                ; Push AX on stack
    push bx                ; Push BX on stack
```



```
pop cx                ; Pop BX into CX
pop dx                ; Pop AX into DX

; Now AX and BX are unchanged, but CX=BX and DX=AX
; Using XCHG to swap CX and DX values

xchg cx, dx          ; Exchange values of CX and DX

push cx
push dx
mov ax, 0x4c00
int 0x21
```

Example 2: Swapping Register Values using Stack

```
[org 0x0100]
mov ax, 1111h
mov bx, 2222h

push ax              ; Push 1111h onto the stack
push bx              ; Push 2222h onto the stack (SP decrements)

pop ax               ; Pop top value (2222h) into AX
pop bx               ; Pop next value (1111h) into BX
mov ax, 0x4c00      ; Terminate program
int 0x21
```

Example 3: Reversing Register Order

```
[org 0x0100]
mov ax, 1
mov bx, 2
mov cx, 3

push ax              ; Stack: [1]
push bx              ; Stack: [2, 1]
push cx              ; Stack: [3, 2, 1]

pop ax               ; AX = 3 (Last In)
pop bx               ; BX = 2
pop cx               ; CX = 1 (First In)
mov ax, 0x4c00
int 0x21
```



Example 4: Inspecting Stack Contents

This task requires you to use the debugger to see *where* in memory the stack lives and what it contains.

```
[org 0x0100]
    mov ax, 0ABCDh
    mov bx, 0FADEh

    push ax
    ; Observe SP. It should have decremented by 2.
    ; Use the memory window to look at address SS:SP.
    ; You should see 'CD' at SS:SP and 'AB' at SS:SP+1.

    push bx
    ; Observe SP. It should have decremented by 2 again.
    ; Look at SS:SP. You should see 'DE' and 'FA'.

    pop cx
    ; Observe SP. It should have incremented by 2.

    pop dx
    ; Observe SP. It should be back to its original value.
mov ax, 0x4c00
int 0x21
```

Example 5: Saving AX and BX

```
[org 0x0100]
jmp start

save_demo:
    push ax
    push bx
    mov ax, 1111h
    mov bx, 2222h
    pop bx
    pop ax
    ret

start:
    mov ax, 9999h
    mov bx, 8888h
    call save_demo
mov ax, 0x4c00
int 0x21
```



Example 6: Safe Subroutine Using PUSH/POP

```
[org 0x0100]
jmp start

calculate:
    push ax
    push bx
    mov ax, [num1]
    add ax, [num2]
    mov [sum], ax
    pop bx
    pop ax
    ret

num1: dw 5
num2: dw 10
sum: dw 0

start:
    mov ax, 1111h
    mov bx, 2222h
    call calculate
mov ax, 0x4c00
int 0x21
```

3. Subroutines & Register Preservation (CALL & RET)

A key use of the stack is managing subroutines (procedures).

- **CALL:** This instruction does two things:
 1. It loads the address of the subroutine into IP to transfer control.
 2. It **pushes the Instruction Pointer (IP)** onto the stack. This is the "return address", the location of the instruction *after* the CALL.
- **RET:** This instruction **pops the return address** from the top of the stack back into IP. This causes execution to resume right after the original CALL instruction.

The Problem: Subroutines often need to use registers (like AX, CX) for their own calculations. This can destroy the caller's data.

The Solution: The subroutine should **save the caller's register values** on the stack upon entry and **restore them** before returning.

Important: Because the stack is LIFO, you must *POP* registers in the **reverse order** that you *PUSHed* them.



Example 7: Subroutine to Calculate Sum

This subroutine takes two numbers in AX and BX, calculates the sum, and returns the result in AX. It uses CX as a temporary "scratch" register, so it must save and restore CX. It demonstrates how a subroutine can use registers internally (like CX) without destroying the caller's data.

```
[org 0x0100]
jmp start

start:
    mov cx, 5555h
    mov ax, 100
    mov bx, 50

    call addNumbers

    ; After function call, CX in the debugger
    ; CX should still be 5555h, even though
    ; addNumbers used it.
mov ax, 0x4c00
int 0x21

addNumbers:
    push cx                ; 1. Save caller's CX

    mov cx, 0              ; Use CX for some internal work
    add ax, bx
    add cx, 1              ; More internal work

    pop cx                 ; 2. Restore caller's CX
    ret
```

Example 8: Nested Subroutines

```
[org 0x0100]
    mov ax, 1111h
    mov bx, 2222h
    call subA
mov ax, 0x4c00
int 0x21

; --- Subroutine A ---
subA:
    push ax                ; Save main AX (1111h)
    push bx                ; Save main BX (2222h)
```



```
    mov ax, 3333h        ; AX for subA use
    mov bx, 4444h        ; BX for subA use
    call subB

    pop bx               ; Restore main BX (2222h)
    pop ax               ; Restore main AX (1111h)
    ret

; --- Subroutine B ---
subB:
    push ax              ; Save subA AX (3333h)
    push bx              ; Save subA BX (4444h)

    mov ax, 5555h        ; AX for subB use
    mov bx, 6666h        ; BX for subB use

    pop bx               ; Restore subA BX (4444h)
    pop ax               ; Restore subA AX (3333h)
    ret
```

Example 9: Nested Subroutines

```
[org 0x0100]
jmp start

sub3:
    mov ah, 3
    mov dl, 'Z'
    ret                  ; Return to sub2

sub2:
    call sub3
    mov ah, 2
    mov dl, 'Y'
    ret                  ; Return to sub1

sub1:
    call sub2
    mov ah, 1
    mov dl, 'X'
    ret                  ; Return to main program

start:
    call sub1
    mov ax, 0x4c00
    int 0x21
```



4. Parameter Passing via Stack

Using registers to pass parameters is fast but limited (you only have so many registers). A more flexible way is to push parameters onto the stack *before* calling the subroutine.

The Problem: When the subroutine starts, the **return address is on top of the stack**. We cannot just POP the parameters, as we would get the return address first.

The Solution: The Base Pointer (**BP**). We will use the BP (Base Pointer) register, which is designed for this. Its default segment is **SS**, so it can be used to "peek" inside the stack without moving the SP.

Standard Subroutine Entry:

mySubroutine:

```
    push bp          ; 1. Save the caller's BP
    mov bp, sp       ; 2. Copy SP to BP. BP is now a stable
```

Accessing Parameters: After the standard entry, the stack looks like this:

- [bp+0]: Holds the old (caller's) BP value.
- [bp+2]: Holds the return address (IP).
- [bp+4]: Holds the *last* parameter pushed by the caller.
- [bp+6]: Holds the *first* parameter pushed by the caller.

Stack Cleanup: *RET n* The parameters pushed by the caller are "waste" after the subroutine returns. The callee can clean them up by using **RET n**, where **n** is the number of bytes to discard. This instruction first pops IP, then adds **n** to SP, effectively removing the parameters. This "*callee-cleanup*" convention is very common.

Example 10: Passing Two Numbers to a Subroutine (*Important*)

```
[org 0x0100]
jmp start
```

add_nums:

```
    push bp
    mov bp, sp
    mov ax, [bp+6]
    add ax, [bp+4]
    pop bp
    ret 4
```

start:

```
    mov ax, 5
    push ax
    mov ax, 10
    push ax
```



```
    call add_nums  
mov ax, 0x4c00  
int 0x21
```

Example 11: Passing Address and Count

```
[org 0x0100]  
jmp start  
  
sum_array:  
    push bp  
    mov bp, sp  
    mov si, [bp+6]    ; start address  
    mov cx, [bp+4]    ; count  
    xor ax, ax  
next:  
    add ax, [si]       ; add current array element to AX  
    add si, 2          ; move to next element  
    loop next         ; decrement CX, jump if not zero  
    pop bp  
    ret 4  
  
array: dw 1,2,3,4,5  
  
start:  
    mov ax, array  
    push ax  
    mov ax, 5  
    push ax  
    call sum_array  
mov ax, 0x4c00  
int 0x21
```

Example 12: Subroutine to Add Two Numbers

```
[org 0x0100]  
jmp start  
  
start:  
    ; We must push parameters in reverse order  
    push 150h          ; First parameter (will be at [bp+6])  
    push 30h           ; Second parameter (will be at [bp+4])  
  
    call addStack      ; Call subroutine  
mov ax, 0x4c00  
int 0x21
```



addStack:

```
    push bp          ; 1. Save old BP
    mov bp, sp       ; 2. Set up stack frame
    push bx          ; 3. Save register we will use

    mov ax, [bp+6]   ; Get first parameter
    mov bx, [bp+4]   ; Get second parameter
    add ax, bx

    pop bx           ; 4. Restore used register
    pop bp           ; 5. Restore old BP
    ret 4             ; 6. Return AND clean up 4 bytes (2 params)
```

Example 13: Subroutine to Find Maximum

```
[org 0x0100]
```

```
jmp start
```

start:

```
    push 900
    push 1200
    call findMax     ; AX should be 1200
```

```
    push 500
    push 100
    call findMax     ; AX should be 500
```

```
    mov ax, 0x4c00
    int 0x21
```

findMax:

```
    push bp
    mov bp, sp

    mov ax, [bp+6]   ; Get first param
    cmp ax, [bp+4]   ; Compare with second param
    jge is_greater   ; If AX >= [bp+4], we are done

    mov ax, [bp+4]   ; Otherwise, second param was larger
```

is_greater:

```
    pop bp
    ret 4
```



5. Lab Tasks

Task 1: Write a program that:

- Moves three values into AX, BX, and CX.
- Pushes them all onto the stack.
- Pops them in reverse order into DX, SI, and DI.
- Exchange DX and SI using XCHG.

Task 2: Write a program that:

- In main, set: AX = 5555h, BX = 2222h, CX = 9999h
- Call a subroutine `modifyRegisters` that:
 - Saves all registers it will use
 - Changes AX = 3333h, BX = 1111h
 - Performs subtraction of AX and BX
 - Restores all registers before returning
- After the call, verify that main's registers are unchanged.

Task 3: Write a program that:

- Write three subroutines: sub1, sub2, sub3
- sub1 calls sub2, and sub2 calls sub3
- Each subroutine prints a unique character ('A', 'B', 'C')

Task 4: Write a subroutine multiply that multiplies two numbers passed via stack:

- Push first number (at [bp+6]) and second number (at [bp+4]).
- In subroutine:
 - Use standard entry (push bp / mov bp, sp).
 - Multiply both values.
 - Return result in AX.
 - Use ret 4 to clean parameters.

Task 5: Write a program that:

- Declare an array of 10 numbers in data segment.
- Pass (start address and count) to a subroutine `sum_array`
- Subroutine should:
 - Use BP to access parameters.
 - Use CX as loop counter and SI to traverse array.
 - Return sum in AX.
 - Restore BP and use ret 4.

Task 6: Write a program that:

- Pushes two values onto stack.



- b) Calls a subroutine **findMin** that:
- Uses BP to access parameters.
 - Compares [bp+6] and [bp+4].
 - Returns the smaller value in AX.
- c) Test with different input pairs.

Task 7: Write a program that:

- a) Modify Task 4 or Task 6 so that the caller clears the stack instead of using RET n.
- Replace ret 4 with ret.
 - After CALL, manually adjust SP (e.g., add sp, 4).
- b) Observe stack pointer values before and after cleanup.

Task 8: Write a program where:

- The main program calls **avg_array**
- **avg_array** calls **sum_array** (using parameters via stack)
- Both subroutines use BP frames and proper stack cleanup

The result should be, AX = average of array elements.